Day 7 Evidence Workbooklet

Ordered Validation and First-True Behavior

Engineering practice to ordered rules to first true branch to skipped branches to support route

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

Name		**Section**		**Date**	
Score	______/42		Mastery check	secure / developing / needs support	

Concrete engineering situation

The sensor-kit checkout table now uses several rules in a fixed order. A kit might have more than one issue, but the route should identify the first action the team should take. Today the focus is ordered validation: trace which condition is first true and explain why later branches are skipped.

Engineering task	Route a sensor kit through an ordered checkout policy without losing evidence about skipped issues.
Computational move	Read an `if`/`elif`/`else` chain as an ordered list of checks where the first true branch wins.
Python focus	`if`, `elif`, `else`, first-true behavior, skipped branches, comparison and string-label checks.
Evidence to keep	rule order, condition result, first true branch, skipped later branch, route assigned, and rule-order risk.

Day 7 working rules

1. Python checks an `if`/`elif`/`else` chain from top to bottom.
2. The first true branch runs.
3. Later branches are skipped after the first true branch.
4. Rule order can change the route.
5. A reliable route note names the first true rule and any important skipped issue.

1 Read the ordered rules before tracing cases

recognize predict

The checkout desk now uses more than one rule. Python checks the rules in order. The first true rule assigns the route and the later rules are skipped.

```
kit_label = "S-22"
charge_percent = 76
calibration_label = "current"
borrower_initials = "MR"
damage_note = ""

if damage_note != "":
    route = "repair bench"
elif charge_percent < 80:
    route = "charge station"
elif calibration_label != "current":
    route = "calibration bench"
elif borrower_initials == "":
    route = "borrower log needed"
else:
    route = "release kit"
```

Pts.	Prompt	My evidence
1	Which rule is checked first?	
1	Which rule is checked second?	
1	Which rule is true for this kit?	
1	Which route is assigned?	

2 Trace first-true behavior

trace explain

Complete the table. Stop at the first true rule for each case.

Pts.	Damage?	Charge	Calibration	Initials	First true rule and route
2	no	76	current	MR	
2	cracked case	76	expired	MR	
2	no	91	expired	MR	
2	no	91	current	blank	

First-true reminder

If an early rule is true, Python does not continue looking for a later, also-true rule. Your evidence should name the first true rule, not just every possible problem.

3 Explain skipped branches

explain debug

A kit may have more than one problem. Ordered routing decides which problem is handled first.

```
charge_percent = 72
calibration_label = "expired"
damage_note = ""

if damage_note != "":
    route = "repair bench"
elif charge_percent < 80:
    route = "charge station"
elif calibration_label != "current":
    route = "calibration bench"
else:
    route = "release kit"
```

Pts.	Prompt	My response
2	Which route is assigned?	
2	Is the calibration rule also true for this kit? Explain.	
2	Why does the calibration branch not run?	
2	What would be dangerous about reporting only the final route without the skipped issue?	

4 Find a rule-order problem

debug test

A teammate proposes moving the release check above the damage check.

```
if charge_percent >= 80 and calibration_label == "current" and borrower_initials != "":
    route = "release kit"
elif damage_note != "":
    route = "repair bench"
else:
    route = "hold for review"
```

Pts.	Prompt	My response
2	What important condition is checked too late?	
2	Give a kit case that would be routed unsafely by the proposed order.	
2	What route should that case receive?	
2	Write one sentence explaining the rule-order risk.	

5 Constrained edit of one rule

implement debug

Only edit the calibration rule below. Do not rewrite the whole route.

```
elif ______________________________________________:
    route = "calibration bench"
```

Pts.	Prompt	My response
3	Complete the calibration rule.	
2	Explain why the rule should compare to `"current"`.	
2	Give one calibration label that should route to the calibration bench.	
1	Name one later branch that would be skipped if this rule is true.	

6 Transfer and support route

transfer explain

Pts.	Ordered-validation note
4	Write a note that explains how ordered validation differs from a single `if/else`. Use first true and skipped branch in your explanation.
2	Circle the support route you would use next: rule order / skipped branch / comparison / string label / written explanation. Name one next action: ______________________

Bridge to Day 8

Day 8 uses expected-vs-actual evidence to debug a route when the selected action does not match the rule intention.